EECE2311 - Summer 2 2025

Lab for EECE2310: A Compact RISC-V Single-Cycle Processor

Lab Assignment 7

EECE2311 Lab Section 01
EECE2310 Course Section 01

Student Name(s): Hang Hang
Student NU ID(s): 002148205

Date Submitted:
08/14/2025

Abstract

This lab implemented a compact single-cycle RISC-V processor capable of executing only R-type instructions. The design leveraged modules from previous lab, including a 4-bit ALU in addition to these register file, instruction memory, and a simplified control unit that decodes only three instruction bits to determine ALU operations. The processor was tested using a 16-instruction RISC-V assembly program, with execution results displayed on 7-segment displays and compared against manual trace calculations.

Introduction

The goal is to implement a simplified single-cycle RISC-V processor that executes a minimal but functionally complete instruction set. This processor represents the fundamental concepts of computer architecture. The target instruction set includes four R-type operations (ADD, SUB, AND, OR) that share the same opcode (0110011), enabling a simplified control unit that requires only three instruction bits for decoding.

The architectural constraints include a 4-bit datapath width, word-addressable instruction memory, and elimination of data memory, branch/jump logic, and immediate value processing. These simplifications allow us to focus on the core processor concepts: instruction fetch, decode, execute, and writeback cycles. The design uses a pushbutton clock for single-step execution. The expected outcome of this lab is a functioning processor that correctly executes RISC-V assembly programs, with results verified through both manual trace analysis and hardware demonstration.

Lab Setup

Pre-Lab

Instructions	x	y	z	ALUControl 1	ALUControl 0
ADD	0	0	0	0	0
	0	0	1	0	0
OR	0	1	0	1	1
AND	0	1	1	1	0
SUB	1	0	0	0	1
	1	0	1	0	0
	1	1	0	0	0
	1	1	1	0	0

Table 1: Truth table for ALUControl 0 and ALUControl 1 generation.

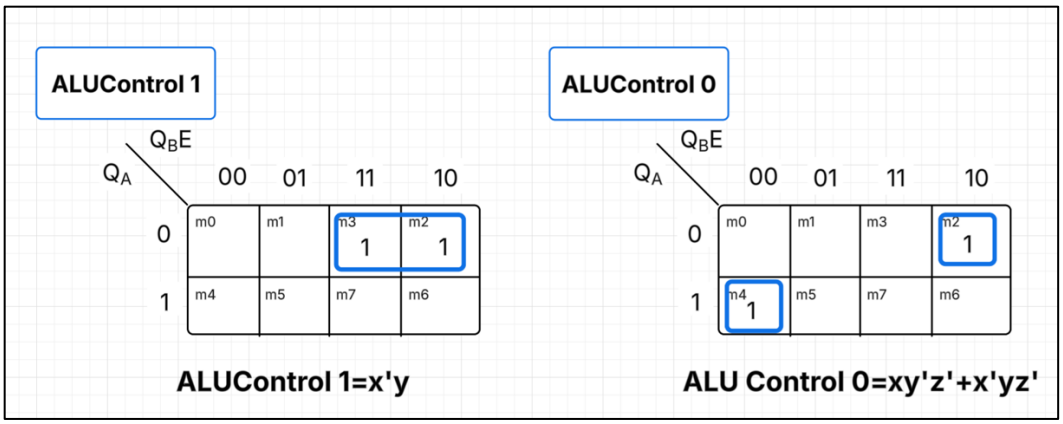

Figure 1: Karnaugh maps and derived Boolean expressions for the two outputs, ALUControl 0 and 1, based on the above truth table.

Equipment

- **Quartus Prime Lite Software**: used for designing, simulating, and validating digital circuit schematics before FPGA hardware implementation.
- **DE1-SoC FPGA board**: a hardware platform containing the Cyclone V FPGA chip used to physically implement and validate the binary adder/subtractor circuits.
- **USB-B to USB-A Cable:** Used to connect the DE1-SoC FPGA board to the lab PC for programming and data transfer.
- **Power Adapter:** Power the DE1-SoC FPGA board.
- **Lab PC:** Run Quartus software and program the board.

Results and Analysis

Results

Assignment 2: Microarchitecture Modules Implementation (Regfile & I-Mem)

```
.text
 main:
    add x0, x0, x0
    add x11, x3, x3
    sub x12, x1, x11
    and x13, x1, x12
    or  x14, x0, x12
```

RISC-V program in RARS

→

```
00000033
003185b3
40b08633
00c0f6b3
00c06733
```

After assembly, instruction's address in hexadecimal code

Assignment 3: The Single Cycle Processor Project

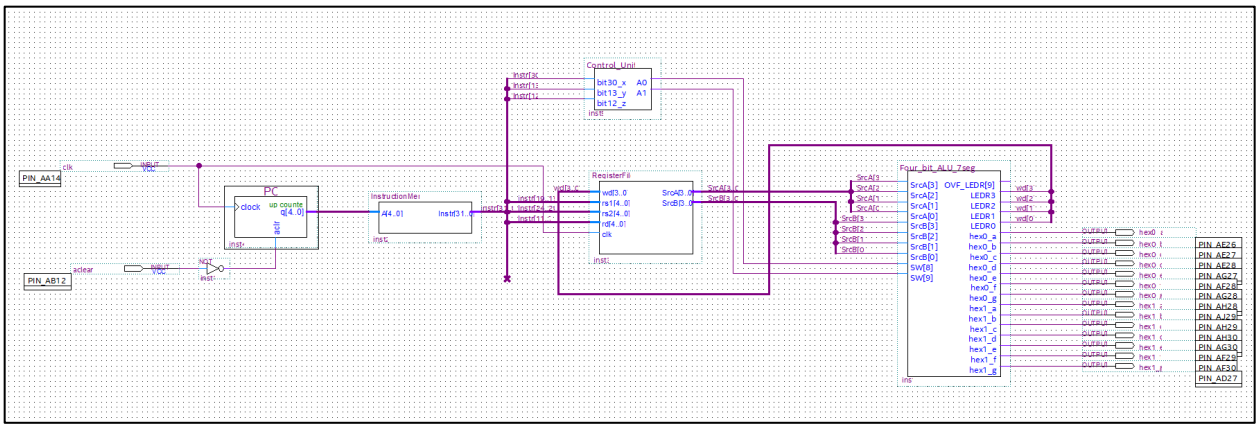

Figure 3: Schematic diagram of single cycle processor (compacted version).

```
    .text
    .globl main
main:
     add x0, x0, x0     # prevents reset change anything, ignored (x0)

     add x5, x1, x2     # x5=3
     sub x6, x3, x1     # x6=2
     and x7, x7, x2     # x7= (x7: 0111 and 0011=0010) AND (x2: 0010 and 0011)=02(0010)
     or  x8, x4, x5     # x8= (x4: 0100 and 0011=0000) OR (x5: 0011 and 0011=0011)= 03(0011)
     add x9, x6, x7     # x9=2+2=4
     sub x10, x9, x1    # x10=4-1=3
     and x11, x9, x6    # x11=(x18: 0010 and 0011=0010) AND (x6: 0010 and 0011=0010)=02(0010)
     or  x12, x11, x3   # x12=(x11: 0000) OR (x3: 0011 and 0010=0011)=03(0011)
     add x13, x10, x12  # x13=3+3=6
     sub x15, x14, x5   # x15=2-3=-1
     and x15, x14, x0   # x15=(x14: 1110 and 0011=0010) AND (x0: 0000 and 0011=0000)= 00
     or  x16, x15, x0   # x16=(x15: 0000) OR (x0: 0000 and 00011=0000)=00
     add x17, x16, x16  # x17=0+0=0
     add x18, x13, x13  # x18=6+6=12 (OVF as the highest representable is 7 or smallest -8)
     or x23, x22, x22   # x23=2(0010) OR 2(0010)=2(0010)
```

Figure 4: The program with new instructions (15 total) and manual traced results (comments), for each instruction, that are stored in the destination register.

Analysis

Assignment 1

- **Bit 13 and 12 in funct3[14:12]**: funct3 indicates ADD/SUB vs. AND/OR operations. When bit 13 and bit 12 in funct3 all = 0, it is either ADD or SUB operation. When funct7=000 and Bit 13=1, bit 12 distinguishes AND vs. OR. Bit 12=1, the op is AND. Bit 12=0, the operation is OR.
- **Bit 30 in funct7[31:25]**: distinguishes ADD vs. SUB operations when funct3 = 000. When bit 30 = 0, the operation is ADD; when bit 30 = 1, the operation is SUB.
- **rs1[19:15]**: 5-bit source register 1 address, specifying the first operand for the ALU operation. This field addresses one of 32 possible registers in the RISC-V regfile.
- **rs2[24:20]**: 5-bit source register 2 address, specifying the second operand for the ALU operation.

- **rd[11:7]**: 5-bit destination register address where the ALU result will be written on the positive clock edge. In our implementation, the register file uses 4-bit data width while maintaining 5-bit addressing for compatibility with the full RISC-V register set (x0-x31).

The simplification to only three instruction bits (30, 13, 12) is possible because:
(1) the opcode is fixed for all R-type instructions
(2) the specific funct3 values (000, 110, 111) gives operation differentiation
(3) bit 30 provides the ADD/SUB distinction within the 000 funct3 group.

Assignment 2

The generated hex code is basically the given instruction in machine code in hexadecimal. For example, for the second instruction:

- Assembly code: add x11, x3, x3
- Binary Machine Code: 0000000 00011 00011 000 01011 0110011
  Funct7 rs2(x3) rs1(x3) funct3 rd(x11) opcode
- Hex Machine Code: 003185b3 (the same machine code but in hex)
- Our processor would read the hex, extracts bits 30, 13, 12 to control the ALU (see highlighted bits).

The five instructions are basically doing the following tasks:
00000033 - NOP (x0 remains 0)
add x11, x3, x3 # 003185b3 - x11 = x3 + x3 = 3 + 3 = 6
sub x12, x1, x11 # 40b08633 - x12 = x1 - x11 = 1 - 6 = -5
and x13, x1, x12 # 00c0f6b3 - x13 = x1 & x12 = 1 & 11 = 01
or x14, x0, x12 # 00c06733 - x14 = x0 | x12 = 0 | 11 = 11

Assignment 3

The manual tracing (see Figure predicts 15 instructions result covering 15+ different registers including register x0. From Registers[i] = i & 2'b11, it creates the pattern:
x0 = 0 x4 = 0 …
x1 = 1 x5 = 1
x2 = 2 x6 = 2
x3 = 3 x7 = 3
This is because the & operator performs bitwise AND between the register number i and the binary value 2'b11 (3 in dec). 2'b is 2-bit binary and 11 means binary 11 = dec 3 = 0011 in 4-bit representation. How this work is that for each register i, we calculate: i & 3. For example, for register 0: 0000 & 0011 = 0000 = 0; Register 1: 0001 & 0011 = 0001 = 1; Register 2: 0010 & 0011 = 0010 = 2, etc. This operation masks the register number to only keep the bottom 2 bits, which creates a repeating pattern 0,1,2,3,0,1,2,3, etc. This offers a predictable 4-bit testing result for each of the 4 type operations.

In terms of the schematic, the processor implements a complete single-cycle datapath: PC generates sequential addresses (0, 1, 2…31) each clock cycle → Instruction Memory outputs 32-

bit instructions and for safety stops loading at 30th address → Control Unit extracts bits [30,13,12] to generate ALUControls → Register File provides operands → ALU performs operations and displays results on 7-segments. The asynchronous clear uses an inverter because the PC counter requires active-low reset.

As for our manually traced results (see Figure 4), the hardware results matched manual calculations. The 16-instruction program successfully exercised all four operations (ADD/SUB/AND/OR) across 15+ registers, proving the processor works correctly.

Conclusion

The design in this lab was achieved through several main aspects including correct execution of R-type instructions, proper datapath operation, and validated control logic functionality. By constraining the instruction set to four operations sharing a common opcode, the control unit complexity was dramatically reduced while preserving essential processor functionality. Future work from this lab could be extended to include immediate-type instructions, branch/jump operations, and data memory access. Additional complexity could involve pipeline implementation as well.

References

[1] Emad Abella, “Lab 7: A Compact RISC-V Single-Cycle Processor”, Northeastern University, August 2025.